

LABORATORIO #2

SNAKE RACE

ARSW

PRESENTADO A: DIEGO ANDRÉS TRIVIÑO GONZALEZ

**REALIZADO POR:
MARIANA MALAGÓN
PAULA LOZANO**

14/08/2026



UNIVERSIDAD

Reporte Parte I (Calentamiento) wait/notify en un programa multi-hilo:

Objetivo

Modificar el programa PrimeFinder para que, cada t milisegundos, se pausen todos los hilos trabajadores, se muestre la cantidad de números primos encontrados hasta ese momento, y el programa espere que el usuario presione ENTER para reanudar la ejecución, todo esto sin espera activa (busy-waiting), usando synchronized, wait() y notify()/notifyAll().

Diseño de sincronización

Monitor compartido. Se creó un único objeto Object lock, instanciado una sola vez dentro del constructor de Control, y pasado por referencia a los tres PrimeFinderThread al construirlos. Es fundamental que sea el mismo objeto para los cuatro hilos (el Control y los tres PrimeFinderThread): si cada hilo tuviera su propio lock, las llamadas a wait() y notifyAll() de uno nunca afectarían a los demás, y no habría forma de coordinar la pausa/reanudación de todos a la vez.

```
private Control() {
    super();
    this.pft = new PrimeFinderThread[NTHREADS];

    int i;

    Object lock = new Object();
    for(i = 0; i < NTHREADS - 1; i++) {
        PrimeFinderThread elem = new PrimeFinderThread(i*NDATA, (i+1)*NDATA, lock);
        pft[i] = elem;
    }
    pft[i] = new PrimeFinderThread(i*NDATA, MAXVALUE + 1, lock);
}
```

```
public PrimeFinderThread(int a, int b, Object lock) {
    super();
    this.primes = new LinkedList<>();
    this.a = a;
    this.b = b;
    this.lock = lock;
}
```

Condición de pausa:

Cada PrimeFinderThread tiene un campo private boolean paused, inicializado en false. Esta variable es la condición que determina si el hilo debe seguir ejecutando o quedar suspendido. Dentro de run(), en cada iteración del ciclo de búsqueda de primos, el hilo evalúa:

```
synchronized (lock) {
    while (paused) {
        try { lock.wait(); }
        catch (InterruptedException lockProblem) { Thread.currentThread().interrupt(); }
    }
}
```

Por qué while y no if:

Se usa while en lugar de if para evitar que el hilo continúe su ejecución basado en una suposición incorrecta del estado. Al despertar de wait() (ya sea por un notifyAll() o por una interrupción), el hilo vuelve a evaluar paused: si sigue siendo true, regresa a dormir, si es false, continúa. Esto garantiza que el hilo nunca "escape" de la pausa mientras la condición siga activa.

Métodos de control externo:

Se agregaron dos métodos públicos en PrimeFinderThread para que Control pueda solicitar el cambio de estado sin acceder directamente a los campos privados:

pause(): dentro de synchronized(lock), pone paused = true. No llama notify, porque en el momento en que se invoca, los hilos aún están ejecutando activamente (no están dormidos todavía), serán ellos mismos quienes, en su siguiente iteración, entren a wait() al detectar paused == true.

```
public void pause(){
    synchronized(lock){
        paused = true;
    }
}
```

resumeExecution(): dentro de synchronized(lock), pone paused = false y llama lock.notifyAll(). Aquí sí es indispensable notificar, porque para este punto los hilos ya están bloqueados en wait(), y wait() no se despierta automáticamente cuando cambia una variable, solo reacciona a una notificación explícita sobre el mismo monitor.

```
public void resumeExecution(){
    synchronized(lock){
        paused = false;
        lock.notifyAll();
    }
}
```

Cómo se evita el lost wakeup:

El riesgo de un lost wakeup ocurre cuando notify() se ejecuta antes de que el hilo receptor llegue a wait(), perdiéndose la señal para siempre. Esto se evita porque:

Toda lectura y escritura de paused, y toda llamada a wait/notifyAll, ocurre siempre dentro de synchronized(lock), el mismo monitor, por lo que no puede haber una intercalación no controlada entre "revisar la condición" y "dormirse".

El uso del while como guardia hace que, incluso si un hilo llega tarde a wait() después de que ya se emitió un notifyAll(), el propio ciclo de vida del programa (pausar -> dormir hilos -> esperar ENTER -> notificar) da tiempo suficiente para que todos los hilos entren en pausa antes de reanudar, ya que Control no continúa hasta que el usuario presiona ENTER.

Ciclo de control (Control.run()). Tras arrancar los tres hilos, Control entra en un bucle que se repite mientras alguno de los PrimeFinderThread siga vivo:

Thread.sleep(TMILISECONDS): espera t milisegundos (bloqueo real del hilo, no consume CPU en un ciclo de verificación).

Llama pause() en cada PrimeFinderThread.

Suma getPrimes().size() de los tres hilos e imprime el total.

Bloquea con Scanner.nextLine() esperando que el usuario presione ENTER.

Llama resumeExecution() en cada PrimeFinderThread.

```

@Override
public void run() {
    for(int i = 0; i < NTHREADS; i++) {
        pft[i].start();
    }

    Scanner scanner = new Scanner(System.in);

    while (anyAlive()) {
        try {
            Thread.sleep(TMILISECONDS);
        } catch (InterruptedException threadProblem) {
            Thread.currentThread().interrupt();
        }

        for (int i = 0; i < NTHREADS; i++) {
            pft[i].pause();
        }

        int total = 0;
        for (int i = 0; i < NTHREADS; i++) {
            total += pft[i].getPrimes().size();
        }
        System.out.println("Primos encontrados hasta ahora: " + total);
        System.out.println(x: "Presiona ENTER para continuar");

        scanner.nextLine();

        for (int i = 0; i < NTHREADS; i++) {
            pft[i].resumeExecution();
        }
    }

    scanner.close();
}

```

Ausencia de espera activa:

En ningún punto del programa se usa un ciclo que consulte repetidamente una condición sin bloquear (por ejemplo, `while(pausado) { }` vacío, o un `Thread.sleep` corto dentro de un ciclo de verificación).

Todas las esperas son bloqueos reales del hilo: `wait()` (liberado solo por `notifyAll()`), `Thread.sleep()` (liberado por el reloj) y `Scanner.nextLine()` (liberado por entrada de teclado).

Evidencia de funcionamiento

Cortando un poco el gran listado que dió:

```

406247
406253
406267
406271
406309
10102481
20070737
Primos encontrados hasta ahora: 44935
Presiona ENTER para continuar

```

Al presionar ENTER:

```

890809
890821
890833
890843
890861
10236137
20141417
Primos encontrados hasta ahora: 93654
Presiona ENTER para continuar

```

Reporte Parte II — SnakeRace concurrente:

1) Análisis de concurrencia

Cómo el código usa hilos para dar autonomía a cada serpiente, esta se logra asignándole un hilo virtual propio. En SnakeApp, se crea un `Executors.newVirtualThreadPerTaskExecutor()` y, por cada Snake de la lista, se somete una instancia de `SnakeRunner` como tarea.

Cada `SnakeRunner` ejecuta un bucle infinito e independiente: decide aleatoriamente si gira (`maybeTurn()`), avanza llamando a `board.step(snake)`, ajusta su velocidad según si está en modo turbo, y duerme (`Thread.sleep`) antes de repetir. Como cada serpiente tiene su propio hilo corriendo este ciclo sin depender de las demás, todas se mueven "por su cuenta" de forma concurrente.

También vimos que existen otros dos hilos relevantes para el análisis de concurrencia:

El hilo EDT (Event Dispatch Thread) de Swing, que pinta la ventana

(`SnakeApp.GamePanel.paintComponent`) y procesa entradas de teclado/botón.

El hilo interno del `GameClock` (`core/engine/GameClock.java`), un `ScheduledExecutorService` que dispara un tick periódico solicitando el repintado al EDT.

Posibles condiciones de carrera

1. `Snake.body`: El cuerpo de la serpiente se guarda en una `ArrayDeque<Position>` `body`. Este campo es modificado por el hilo `SnakeRunner` correspondiente a través de `advance()` (llamado indirectamente desde `board.step(snake)`), y leído por el hilo EDT a través de `snapshot()`, invocado desde `GamePanel.paintComponent()` en cada repintado. Ambos accesos ocurren sin ninguna sincronización, por lo que dos hilos distintos pueden tocar la misma estructura al mismo tiempo.
2. `Snake.direction`: El método `turn()` realiza una operación de verificar y luego escribir (revisa si la nueva dirección es opuesta a la actual antes de asignarla) que no es atómica como conjunto. `turn()` puede ser invocado tanto por el hilo EDT (cuando el jugador presiona una tecla) como por el propio hilo `SnakeRunner` (giros aleatorios). Si ambos hilos entran a `turn()` casi al mismo tiempo, el resultado depende del orden de ejecución, lo cual es una condición de carrera de menor severidad (no revienta el programa, pero puede producir un giro inconsistente).

Colecciones o estructuras no seguras en contexto concurrente

`Snake.body` (`ArrayDeque`): colección no seleccionada para concurrencia y, a diferencia del resto del código, sin ninguna protección (`snapshot()`, `head()` y `advance()` no usan `synchronized`). Este es el punto más frágil encontrado.

`Board.mice`, `Board.obstacles`, `Board.turbo` (`HashSet`) y `Board.teleports` (`HashMap`): también son colecciones inherentemente no seguras para concurrencia. Pero, en el código actual sí están protegidas, porque todos los métodos públicos que las tocan como `mice()`, `obstacles()`, `turbo()`, `teleports()` o `step()` están declarados `synchronized` sobre el mismo monitor (la instancia de `Board`). Esto evita el mismo problema que sí ocurre en `Snake`. Es un punto a favor del diseño actual de `Board`, pero también una fragilidad a vigilar, ya que si en el futuro se agregara un nuevo método que accediera a estas colecciones sin `synchronized`, se rompería la protección silenciosamente.

Ocurrencias de espera activa (busy-wait) o sincronización innecesaria

No encontramos ocurrencias de espera activa en el código actual, porque SnakeRunner usa Thread.sleep(sleep) entre iteraciones, la cual es una espera pasiva (el hilo libera el CPU), no un while consultando una condición en bucle.

GameClock usa ScheduledExecutorService.scheduleAtFixedRate junto con un AtomicReference<GameState>, evitando también cualquier tipo de polling.

2) Correcciones mínimas y regiones críticas

Cambio realizado en sincronización de Snake.java:

Agregamos el modificador synchronized a los tres métodos de Snake que acceden al campo body (ArrayDeque<Position>): head(), snapshot() y advance().

```
public synchronized Position head() { return body.peekFirst(); }

public synchronized Deque<Position> snapshot() { return new ArrayDeque<Position>(body); }

public synchronized void advance(Position newHead, boolean grow) {
    body.addFirst(newHead);
    if (grow) maxLength++;
    while (body.size() > maxLength) body.removeLast();
}
```

Riesgo que resolvía: body era leído por el hilo EDT de Swing (a través de snapshot()), invocado en cada repintado desde GamePanel.paintComponent() y modificado concurrentemente por el hilo SnakeRunner de esa serpiente (a través de advance(), invocado desde board.step(snake)), sin ninguna sincronización.

Cómo los resolvimos: al declarar los tres métodos como synchronized, Java los sincroniza automáticamente sobre this (la instancia de Snake). Como tanto el hilo SnakeRunner como el hilo EDT operan sobre la misma referencia de Snake, ese this actúa como monitor compartido entre ambos, garantizando exclusión mutua sobre body sin necesidad de crear un lock adicional.

Alcance de la región crítica: se mantuvo mínimo a propósito, ya que, cada método sincronizado realiza únicamente la operación estrictamente necesaria sobre body (una lectura, una copia, o una inserción/eliminación), sin abarcar lógica adicional ni bloquear el objeto por más tiempo del indispensable.

Decisión de no sincronizar turn()/direction():

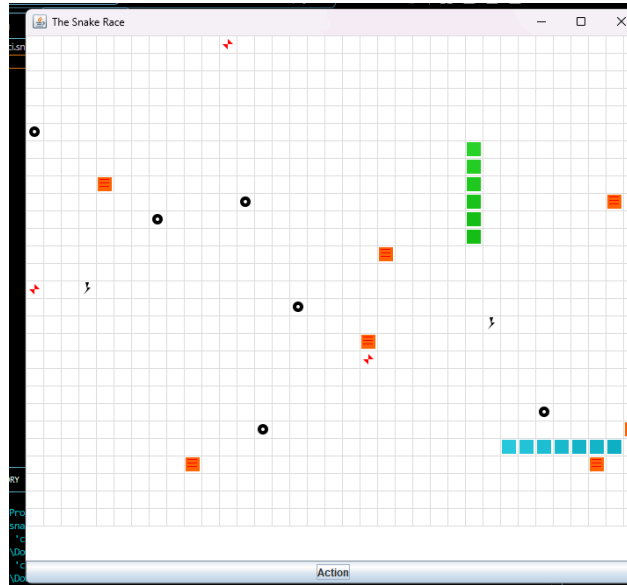
Se decidió no ampliar la región crítica para cubrir este caso, porque la consecuencia de esa carrera es de severidad baja (un giro ocasionalmente inconsistente, sin riesgo de excepción ni corrupción de datos), y el enunciado pide proteger solo las regiones críticas estrictamente necesarias, evitando bloqueos amplios innecesarios.

Confirmamos que Board ya protegía adecuadamente sus colecciones (mice, obstacles, turbo, teleports): todos los métodos públicos que las exponen o modifican (mice(), obstacles(), turbo(), teleports(), step()) ya estaban declarados synchronized sobre la misma instancia, por lo que no requería corrección.

Espera activa: sin ocurrencias que eliminar.

Como se documentó en el análisis, no se encontraron instancias de busy-waiting en el código base (SnakeRunner usa Thread.sleep, GameClock usa ScheduledExecutorService), por lo que no hubo nada que reemplazar en este punto.

Prueba de funcionalidad sin problemas hasta el momento:



3) Control de ejecución seguro (UI)

Decisión de diseño: regla de "muerte":

El enunciado nos pide mostrar "la peor serpiente (la que primero murió)", pero el código original no tenía ningún concepto de muerte: los obstáculos solo producían rebote. Decidimos implementar la regla clásica de Snake: una serpiente muere si su cabeza colisiona con su propio cuerpo. Es la interpretación más natural del juego, no contradice ninguna regla existente (los obstáculos siguen funcionando igual, por rebote), y permite tener un criterio objetivo y verificable de "quién murió primero".

```
public synchronized MoveResult step(Snake snake) {
    Objects.requireNonNull(snake, message: "snake");
    var head = snake.head();
    var dir = snake.direction();
    Position next = new Position(head.x() + dir.dx, head.y() + dir.dy).wrap(v

    if (obstacles.contains(next)) return MoveResult.HIT_OBSTACLE;

    boolean teleported = false;
    if (teleports.containsKey(next)) {
        next = teleports.get(next);
        teleported = true;
    }

    if (snake.snapshot().contains(next)) {
        snake.kill();
        return MoveResult.DIED;
    }
}
```

Cuando una serpiente muere, su propio hilo SnakeRunner termina su bucle (break) y deja de ejecutarse, la serpiente queda congelada en el tablero permanentemente, y ya no responde a las teclas del jugador (si era la controlada por el usuario), porque board.step() nunca vuelve a invocarse para ella.

Pausa real:

En el análisis inicial se identificó que el GameClock.pause() original solo detenía el repintado, pero los hilos SnakeRunner seguían moviendo a las serpientes en segundo plano. Para corregir esto, se implementó una pausa real usando el mismo patrón synchronized/wait/notifyAll de la Parte I, esta vez sobre Board (el objeto que ya comparten todos los SnakeRunner):

Board.pauseSnakes(int activeRunners): marca un flag paused = true.

SnakeRunner, al inicio de cada iteración de su bucle, llama a board.awaitIfPaused(), que revisa el flag dentro de un while(paused) { wait(); } y se bloquea si corresponde.

Board.resumeSnakes(): pone paused = false y llama notifyAll(), despertando a todos los hilos bloqueados.

```
public synchronized void pauseSnakes(int activeRunners) {
    paused = true;
    pauseConfirmations = new CountDownLatch(activeRunners);
}

public synchronized void resumeSnakes() {
    paused = false;
    notifyAll();
}

public synchronized void awaitIfPaused() throws InterruptedException {
    if (paused) {
        pauseConfirmations.countDown();
        while (paused) {
            wait();
        }
    }
}
```

```
@Override
public void run() {
    try {
        while (!Thread.currentThread().isInterrupted()) {
            board.awaitIfPaused(); ←
            maybeTurn();
            var res = board.step(snake);
            if (res == Board.MoveResult.DIED) {
                break;
            } else if (res == Board.MoveResult.HIT_OBSTACLE) {
                randomTurn();
            } else if (res == Board.MoveResult.ATE_TURBO) {
                turboTicks = 100;
            }
            int sleep = (turboTicks > 0) ? turboSleepMs : baseSleepMs;
            if (turboTicks > 0) turboTicks--;
            Thread.sleep(sleep);
        }
    } catch (InterruptedException ie) {
        Thread.currentThread().interrupt();
    }
}
```


Evitando el "tearing" al mostrar estadísticas

Como la suspensión de N hilos no es instantánea (cada SnakeRunner solo revisa el flag de pausa en su propio ritmo, hasta cada 80ms), calcular las estadísticas inmediatamente después de pedir la pausa podría capturar un estado a medias, con algunas serpientes ya congeladas y otras todavía en movimiento.

Para resolverlo usamos un CountdownLatch, inicializado cada vez que se pausa con el número de serpientes actualmente vivas:

Cada SnakeRunner, justo antes de bloquearse en wait(), baja el contador (pauseConfirmations.countDown()), una confirmación como de "ya estoy congelado".

La UI, tras pedir la pausa, llama board.awaitAllPaused(500), que bloquea hasta que el contador llegue a 0 (todas confirmaron) o hasta un tope de 500ms (para no colgarse indefinidamente si, por ejemplo, una serpiente muere justo en ese instante y nunca llega a confirmar).

Solo después de esa confirmación se calculan las estadísticas, garantizando una "foto" coherente de un mismo instante.

UI: Iniciar / Pausar / Reanudar

El botón (antes fijo entre "Action"/"Resume", que solo controlaba el repintado) ahora es una máquina de tres estados:

Iniciar: arranca los hilos SnakeRunner (antes se lanzaban automáticamente en el constructor) y el GameClock.

Pausar: ejecuta la secuencia descrita arriba (pausar, esperar confirmación, calcular y mostrar estadísticas con Comparator.comparingInt(Snake::length) para la más larga viva, y Comparator.comparingLong(Snake::deathOrder) para la primera en morir) en un diálogo (JOptionPane).

Reanudar: llama board.resumeSnakes() y clock.resume().

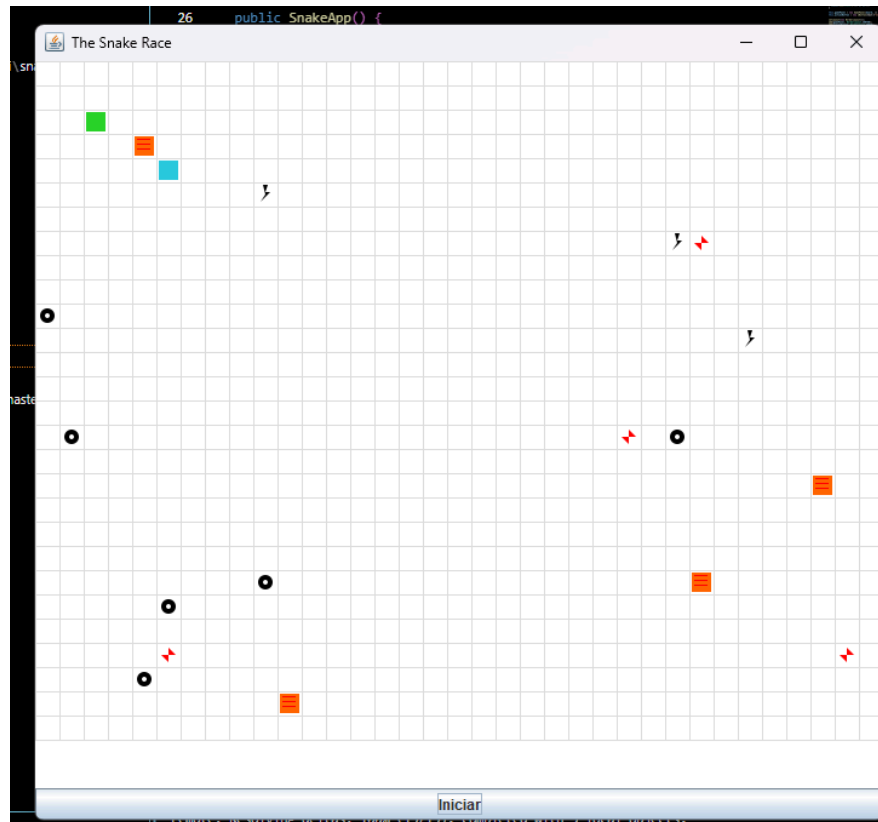
```
private void onAction() {
    switch (actionButton.getText()) {
        case "Iniciar" -> {
            startGame();
            actionButton.setText(text: "Pausar");
        }
        case "Pausar" -> {
            pauseAndShowStats();
            actionButton.setText(text: "Reanudar");
        }
        default -> {
            board.resumeSnakes();
            clock.resume();
            actionButton.setText(text: "Pausar");
        }
    }
}
```

Decisión consciente documentada: pauseAndShowStats() se ejecuta en el hilo EDT (el del botón/teclado), y el awaitAllPaused(500) bloquea ese hilo hasta 500ms mientras espera la confirmación de todas las serpientes. Esto congela brevemente la ventana al pausar. Se aceptó este trade-off por simplicidad, dado que el bloqueo es corto y predecible; una alternativa más sofisticada

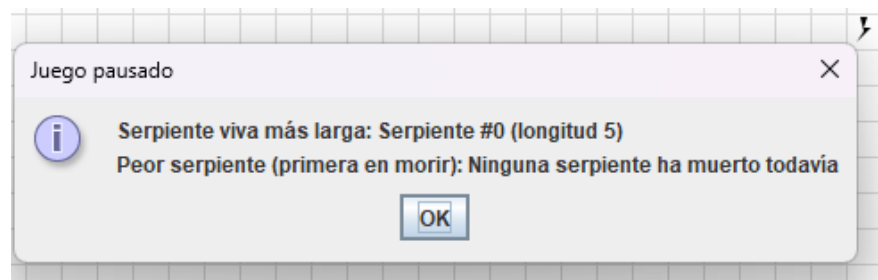
sería mover esta espera a un hilo aparte y actualizar la UI de forma asíncrona con `SwingUtilities.invokeLater`, pero se consideró innecesario para el alcance de este laboratorio.

Evidencia de funcionamiento

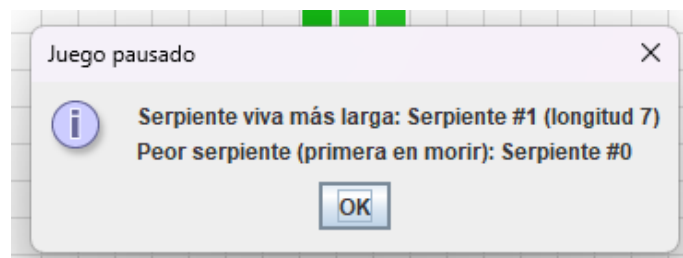
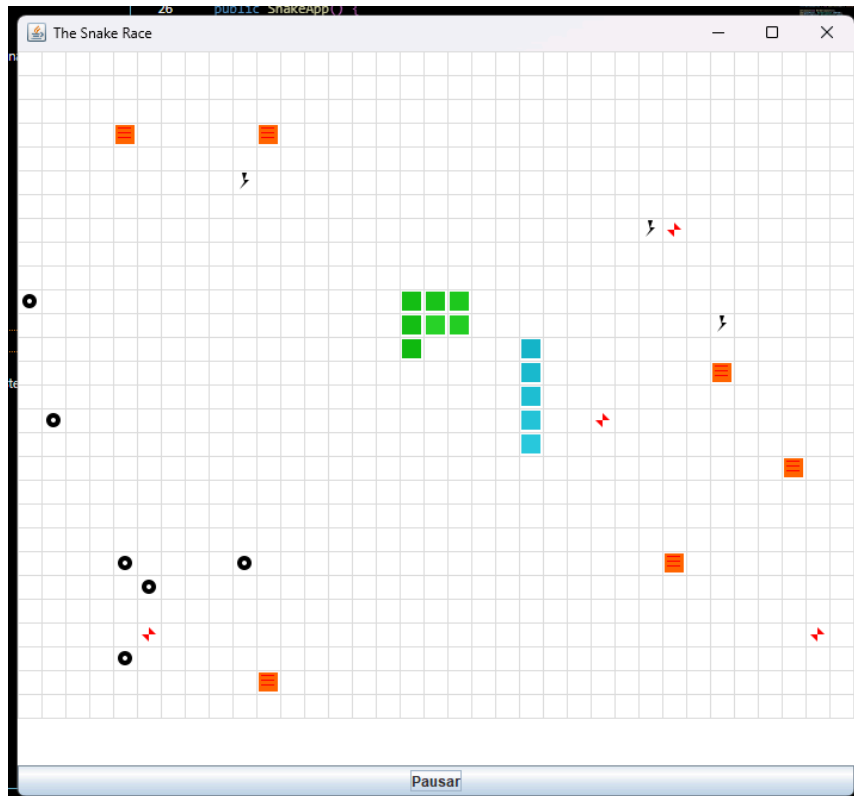
Estado inicial:



Pausado cuando ninguna serpiente a muerto:



Acá murió la serpiente verde entonces quedó congelada:



4) Robustez bajo carga

Un vídeo mostrando la funcionalidad con 20 serpientes:

[Video Snake Race](#)

<https://youtu.be/b00DTXKqEgQ>

Conclusiones:

Se ejecutó el juego con `-Dsnakes=20` durante varios minutos, incluyendo los siguientes escenarios de estrés:

Ejecución prolongada con teletransportadores y turbo habilitados (activos por defecto: 2 pares de teletransportadores y 3 ítems de turbo generados al inicio).

Múltiples muertes por colisión (propia) y consumo de ratones ocurriendo simultáneamente entre distintas serpientes.

Pausas y reanudaciones repetidas y consecutivas, incluyendo el caso límite de pausar cuando ya no quedaban serpientes vivas.

Resultado: el juego se mantuvo estable en todos los escenarios. No se observaron excepciones en consola (`ConcurrentModificationException` u otras), ni lecturas visualmente inconsistentes (posiciones corruptas, parpadeos erráticos del cuerpo de las serpientes), ni bloqueos (deadlocks), el botón de pausa/reanudar respondió correctamente incluso tras pausar repetidamente en rápida sucesión.

```
PS C:\Users\maria\Downloads\ARSW_Lab_SnakeRace> java -Dsnakes=20 -cp out co.eci.snake.app.Main
```

Esto confirma en la práctica las correcciones aplicadas en las secciones anteriores:

La sincronización de Snake (`head()`, `snapshot()`, `advance()`) evita la corrupción de body que antes era posible entre el hilo `SnakeRunner` y el hilo EDT, incluso bajo la carga de 20 hilos repintando y moviéndose simultáneamente.

La protección ya existente de Board (colecciones `mice`, `obstacles`, `turbo`, `teleports` bajo `synchronized`) soportó sin problemas la frecuencia de mutación adicional que implica un N alto.

El mecanismo de pausa con `CountDownLatch` no se bloqueó ni siquiera en el caso límite de 0 serpientes vivas (el `await` con `timeout` de 500ms evita que la UI se cuelgue si el contador nunca llega a completarse).